

Chance constrained programs with Gaussian mixture models

Zhaolin Hu, Wenjie Sun & Shushang Zhu

To cite this article: Zhaolin Hu, Wenjie Sun & Shushang Zhu (2022) Chance constrained programs with Gaussian mixture models, IIE Transactions, 54:12, 1117-1130, DOI: 10.1080/24725854.2021.2001608

To link to this article: <https://doi.org/10.1080/24725854.2021.2001608>



View supplementary material [↗](#)



Published online: 04 Jan 2022.



Submit your article to this journal [↗](#)



Article views: 781



View related articles [↗](#)



View Crossmark data [↗](#)



Citing articles: 3 View citing articles [↗](#)



Chance constrained programs with Gaussian mixture models

Zhaolin Hu^a , Wenjie Sun^b , and Shushang Zhu^c 

^aSchool of Economics and Management, Tongji University, Shanghai, China; ^bDepartment of Analytics and Operations, NUS Business School, National University of Singapore, Singapore; ^cDepartment of Finance and Investment, School of Business, Sun Yat-Sen University, Guangzhou, China

ABSTRACT

In this article, we discuss input modeling and solution techniques for several classes of Chance constrained programs (CCPs). We propose to use a Gaussian Mixture Model (GMM) to fit the data available and to model the randomness. We demonstrate the merits of using a GMM. We consider several scenarios that arise from practical applications and analyze how the problem structures could embrace alternative optimization techniques. More specifically, for several scenarios, we study how to assess the gradient of the chance constraint and incorporate the results into gradient-based nonlinear optimization algorithms, and for a class of CCPs, we propose a spatial branch-and-bound procedure and solve the problems to global optimality. We also conduct numerical experiments to test the efficiency of our approach and propose an example of hedge fund portfolio to illustrate the practical application of the method.

ARTICLE HISTORY

Received 6 April 2021
Accepted 12 October 2021

KEYWORDS

Stochastic programming; chance constrained programs; Gaussian mixture model; spatial branch-and-bound; value-at-risk

1. Introduction

Stochastic Optimization (SO) has become a standard modeling philosophy and framework for decision making under uncertainty. Chance constrained programs (CCPs), as a basic class of SO models, have been used in various areas. When there are random parameters in the constraints of an optimization problem, it is natural to require the constraints be satisfied with a given high probability and formulate the problem as a CCP. The literature on CCPs can be dated back to Charnes *et al.* (1958), who first considered a single CCP (SCCP), and Miller and Wagner (1965), who first considered a joint CCP (JCCP). Since then, both theories and applications of CCPs have been studied extensively. For a comprehensive study and literature review on the topic, readers are referred to Prékopa (2003), Nemirovski and Shapiro (2006), Hong *et al.* (2011), Zymler *et al.* (2013), Hanasusanto *et al.* (2017), Geng and Xie (2019) and references therein.

In the framework of SO, the randomness of the system is modeled by random parameters. The process of specifying a distribution for the random parameters is often called input modeling. Input modeling may directly affect the optimization results, and thus affect the decision. In practice, there is usually no distribution that is ready for use, and decision makers need to conduct input modeling based on the data/information available. In this article, we make the following emphasis: Since the optimal solutions of CCPs critically depend on the input distributions, input modeling is a critical module for CCPs, and is perhaps equally important as the

solution techniques. We conduct an integrated study between the input modeling module and the optimization module.

Input modeling has been extensively studied in the area of simulation, see, e.g., Law (2013). When a simulation model is being built, the modeler wants to specify the input distribution for the random parameters that drive the stochastic system. There have been various approaches for input modeling in simulation. When there is data, one can fit a distribution to that data. When there is no data, the modeler often needs to specify the distribution based on some subjective judgement. In simulation, the data-based approaches can roughly be divided into the parametric approach and the nonparametric approach. In the parametric approach, decision makers select a parametric distribution family, e.g., a multivariate normal (Gaussian) distribution, and then use statistical methods to determine (estimate) the parameters of the distribution, e.g., the mean vector and the covariance matrix. In the nonparametric approach, decision makers do not specify a parametric family. Rather, they use some nonparametric methods to fit the distribution. For instance, the kernel density estimation methods, which can be dated back to Rosenblatt (1956) and Parzen (1962), are often used in practice.

Nowadays we are witnessing an era of data. In many real SO problems, we can obtain data for the random parameters. Thus, it is often beneficial to mine the data to specify the input distribution for SO. This kind of data-driven approach is very natural and attractive. In this article we adopt such an input modeling approach for CCPs. We propose to use mixture distributions (also called mixture

models) to model the randomness of the parameters. Note that a mixture distribution is a weighted sum of a finite set of probability measures. In contrast with the parametric approach and the nonparametric approach, the use of a mixture distribution can be viewed as a semiparametric approach (McLachlan and Peel, 2000; Bishop, 2006). The idea of using a mixture distribution can be dated back to Pearson (1894), who proposed to use a mixture of two normal distributions to fit a set of asymmetric biological data. Pearson's approach led to new findings in the biological area. Mixture models allow for sufficient flexibility (see, e.g., McLachlan and Peel (2000) and Frühwirth-Schnatter (2006)). They provide a framework to construct more complex distributions. For instance, asymmetrical distributions and skewed distributions can often be closely approximated by mixture models (see, e.g., Wang and Taaffe, 2015). Among various mixture distributions, we are particularly interested in the Gaussian Mixture Model (GMM), which has been studied extensively in the literature, and has been used widely in various areas, including economics, finance, engineering and social sciences.

Recently a GMM has been used in distributionally robust CCPs. For instance, Chen *et al.* (2018) considered a GMM-based distributionally robust CCP, where they assumed the weights of the GMM belonged to some uncertainty set. They proposed lower and upper approximations to their model under certain conditions. Lasserre and Weisser (2021) investigated a more general mixture setting. They built inner approximation sets constructed from polynomials to the feasible region of their distributionally robust CCP. Instead of exploring robust optimization, our work studies conventional CCPs. We use GMM to conduct input modeling (to learn the underlying distribution), and propose optimization procedures (gradient-based method and a spatial branch-and-bound algorithm) to directly solve different classes of CCPs under the learned GMM. By applying mixture input models in CCPs, we show that our approach has many merits and can extend the current research results for CCPs in some aspects.

It is well known that CCPs are in general difficult to solve. Modelers often need to consider special structures of the CCPs in order to handle them. In this article, we consider several scenarios that arise from real applications and discuss how the chance constraints can be simplified accordingly. As a straightforward, but important, result, we show that the probability function in a CCP with a mixture distribution can be transformed as a weighted sum of probability functions with component distributions of the mixture distribution. Based on the result, we analyze a class of JCCPs and a class of SCCPs.

For JCCPs, we consider a class of CCPs in which the constraints within the probability function have a linear structure, and the random vector and the decision vector are separated. Many engineering management problems can be modeled as such CCPs. The electricity network capacity optimization problem studied by Henrion and Möller (2012) and the renewable energy management problem studied by Bremer *et al.* (2015) are typical examples. When the underlying distribution is a (possibly singular) Gaussian distribution, certain

results about the gradient of the probability function have been derived. For instance, Henrion and Möller (2012) provided some explicit formulas for the probability function, which allows one to reduce the calculation of gradients to the calculation of probability function values. Nevertheless, it turns out to be difficult to build similar results for other (parametric) distribution families. In this article, we show that the results can be easily generalized for the GMM. For SCCPs, we consider a class of CCPs where the constraint within the probability function takes a product form of the randomness and the decision vector. This class of models has found applications in various areas. We show that for the GMM, the problems can be reformulated as deterministic optimization problems where the standard normal distribution function plays a substantial role. Therefore, we can directly use conventional nonlinear optimization techniques to handle the problems. Note that the nonlinear optimization techniques may not guarantee global optimality. In this article, we further explore the problem structure and propose a globally convergent spatial branch-and-bound (B&B) algorithm to solve this class of CCPs.

The main contributions of this article are summarized as follows:

1. We consider the integration of the input modeling module and the solution module when using CCPs in practice, and highlight the importance of such an integration. This integrated approach can benefit management practice.
2. We propose to use GMM to model the random distribution in the CCP. GMM provides sufficient freedom to learn the data information.
3. We explore structures and properties for a number of CCPs under the GMM framework, and develop efficient optimization methods to solve different classes of CCPs. More specifically, for several scenarios, we study how to assess the gradient of the chance constraint under GMM and incorporate the results into gradient-based nonlinear optimization algorithms, and for a class of CCPs, we propose a spatial B&B procedure and solve the problems to global optimality.

The rest of this article is organized as follows. In [Section 2](#) we introduce the mixture models and analyze the CCPs with mixture models. In [Section 3](#) we consider a number of scenarios and demonstrate the potential wide applications of our approach. We conduct a numerical study in [Section 4](#). [Section 5](#) concludes this article. Some more materials are included in the Online Supplement.

2. Mixture distributions and model structures

We start from the general CCP paradigm:

$$\underset{x \in X}{\text{minimize}} \quad h(x) \quad (1a)$$

$$\text{subject to } \Pr_{\sim P_s} \{c_1(x, \xi) \leq 0, \dots, c_m(x, \xi) \leq 0\} \geq 1 - \alpha. \quad (1b)$$

In Problem (1a)-(1b), x is the decision vector, which belongs to a set $X \subset \mathbb{R}^d$, $h : \mathbb{R}^d \rightarrow \mathbb{R}$ is a real-valued function which models the objective, ξ is a k -dimensional vector of random parameters and the support of ξ , denoted as Ξ , is a closed subset of $\mathbb{R}^k$, $c_i : \mathbb{R}^d \times \Xi \rightarrow \mathbb{R}, i = 1, \dots, m$ are real-valued functions, and the notation $\Pr_{\sim P_*}$ denotes that the probability is taken with respect to P_* , where P_* is referred to as the underlying distribution. Constraint (1b) is called a chance constraint. It requires that the m constraints are satisfied at least with a probability $1 - \alpha$. Problem (1a)-(1b) is called a SCCP if $m = 1$ and a JCCP if $m > 1$. This is a very natural requirement in mathematical programs when random parameters are involved in the constraints. Throughout this article we assume that X is a convex and compact set, which may be defined by some deterministic constraints, and the function h can be handled by deterministic optimization techniques. Thus, the major difficulty of solving Problem (1a)-(1b) comes from the chance constraint (1b).

Although the idea of CCP is conceptually very natural and fits decision making very well, solving a CCP is often a non-trivial task. In general, CCPs are challenging problems. There are a number of reasons for this situation. For instance, the probability function in a CCP typically has no closed form, and the convexity of the probability function is not easy to preserve. Many approaches have been developed to tackle CCPs. A natural approach is to treat the probability function as a nonlinear function and use nonlinear optimization techniques (see, e.g., Henrion and Möller (2012) and Bremer *et al.* (2015)). In addition, another two streams of approaches are widely used. One stream can be termed as a deterministic optimization approach. In this approach, a deterministic optimization model is constructed to approximate the CCP. That deterministic model is then solved and the solution is used to approximate that of the CCP. It includes the tractable safe approximations and the robust optimization framework, which have been explored extensively in the literature, see, e.g., Nemirovski and Shapiro (2006) and Chen *et al.* (2010). Another stream is to use a sampling-based approach. In that approach, the sample used is simulated from a specified distribution or is obtained from historical data. Given the sample, a sample problem is constructed associated with the CCP and different optimization procedures are then designed to solve the sample problem. These include the Sample Average Approximation (SAA) approach (see, e.g., Luedtke and Ahmed (2008), Pagnoncelli *et al.* (2009) and Luedtke *et al.* (2010)), the scenario approach (Calafiore and Campi, 2005, 2006), the p -efficient point-based approach (Dentcheva *et al.*, 2000), and the binarization approach (Lejeune, 2012). In the simulation framework, the CCP is often treated or transformed as a nonlinear optimization problem, and simulation-based methods are developed to solve it, see, e.g., Hong *et al.* (2011), Henrion (2013), Hu *et al.* (2013) and Peña-Ordieres *et al.* (2020).

2.1. Probability function with mixture distribution

When Problem (1a)-(1b) is being studied, the underlying distribution P_* is an important module. However, as

discussed in Section 1, in practice, such a distribution is usually not pre-given exactly and can only be inferred. In this article, we model that P_* has the following density

$$p_*(z) = \sum_{j=1}^K \pi_j p_j(z) \quad (2)$$

with the Lebesgue measure, where $p_j(z), j = 1, \dots, K$ are probability densities that are called component distributions, and $\pi_j > 0, j = 1, \dots, K$ are mixing coefficients satisfying $\sum_{j=1}^K \pi_j = 1$. The distribution with density (2) is called a mixture distribution (or mixture model), see, e.g., McLachlan and Peel (2000). Note that throughout the analysis we do not differentiate the notations P_* and p_* when there is no confusion.

2.2. GMM

A well known instance of the mixture models is the GMM. Following the notion in Bishop (2006), a GMM has the following density:

$$p_*(z) = \sum_{j=1}^K \pi_j \mathcal{N}(z | \mu_j, \Sigma_j), \quad (3)$$

where each component $\mathcal{N}(z | \mu_j, \Sigma_j)$ is a (multivariate) normal density function with mean vector μ_j and covariance matrix Σ_j . Throughout this article we assume that $\Sigma_j, j = 1, \dots, K$ are positive definite. The GMM is widely used in the machine learning literature, see, e.g., Bishop (2006). Using GMM to approximate a distribution is very popular in the literature, see, for instance, Zeevi and Meir (1997), Li and Barron (2000) and Maugis-Rabusseau and Michel (2013). Wilson and Calway (2001) showed that the GMM can approximate a given distribution to any accuracy under some metric. Recently, GMMs have also been used to learn the response surface and to guide the random search in simulation optimization, see, e.g., Sun *et al.* (2018). In practice, the distribution of the random parameters in a CCP may be skewed, asymmetrical, or multimodal. These distribution structures are typically hidden in the data that are collected. Consequently, conventional parametric families, e.g., a Gaussian distribution or an exponential family may not closely fit the data. In such situations, it may be better to use a mixture distribution to model the randomness.

As indicated in the literature, there are several issues surrounding the use of mixture models. One key issue is how to fit the mixture models to the data. There are a number of approaches that have been developed for such fitting procedures. A widely used method is the Expectation Maximization (EM) method, see, for instance, Bishop (2006). In Online Supplement 1 we provide detailed procedures for EM.

2.3. Model structures

We now explore structures and properties of constraint (1b) when the underlying distribution is a mixture distribution. Let $c(x, \xi) = \max\{c_1(x, \xi), \dots, c_m(x, \xi)\}$. Then constraint (1b) can be rewritten as $\Pr_{\sim P_*} \{c(x, \xi) \leq 0\} \geq 1 - \alpha$. Suppose

that the distribution P_* is a mixture distribution with a density defined by (2). Then

$$\Pr_{\sim P_*} \{c(x, \xi) \leq 0\} = \sum_{j=1}^K \pi_j \Pr_{\sim P_j} \{c(x, \xi) \leq 0\}. \quad (4)$$

To see this, we consider a simple property of a GMM. Let $H(\xi)$ be a function of ξ . Suppose the expectation of $H(\xi)$ under each $P_j, j = 1, \dots, K$ exists. Then

$$\begin{aligned} E_{P_*}[H(\xi)] &= \int H(z) \sum_{j=1}^K \pi_j p_j(z) dz = \sum_{j=1}^K \pi_j \int H(z) p_j(z) dz \\ &= \sum_{j=1}^K \pi_j E_{P_j}[H(\xi)], \end{aligned}$$

where the first and third equations follow from the definition of expectation, and the second equation holds since we can interchange the order of the integral and finite sum. Then (4) follows from the above equation by setting $H(\xi) = \mathbb{1}_{\{c(x, \xi) \leq 0\}}$, which equals 1 if $c(x, \xi) \leq 0$ and 0 otherwise.

The formula (4) shows that the probability function with a mixture distribution can be expressed as the summation of probability functions with the underlying distributions being the component distributions. This result, from a crude simulation perspective, does not simplify the problem. However, it has interesting applications when each term in the summation can be simplified or handled efficiently. We will provide concrete examples to support the statement in the next section.

Problem (1a)-(1b) is a nonlinear optimization problem. Under the simulation-based optimization framework, a natural approach to solve the problem is to use nonlinear optimization techniques. A nonlinear optimization algorithm typically requires the function values and the function gradients of the objective and constraints. For CCPs, these values are often difficult to derive analytically, and thus, they need to be estimated. In the literature, various methods have been proposed to estimate the gradient of the probability function, see, for instance, Prékopa (2003), Henrion (2012) and Henrion and Möller (2012). The difficulties of the estimation procedures, of course, depend on the structures of the probability function. However, in general, estimating the gradient of a probability function is a challenging problem. In this article, we analyze the gradient estimation for the chance constraint (1b). We now consider a GMM with density (3). We make the following assumption.

Assumption 1. $\Pr_{\sim P_j} \{c(x, \xi) \leq 0\}, j = 1, \dots, K$ are continuously differentiable with respect to x where $x \in X$.

Let $F_j(t, x)$ denote the cumulative distribution function of $c(x, \xi)$ under P_j . Suppose that $F_j(t, x)$ is continuously differentiable in $(t, x) \in (-\delta_j, \delta_j) \times X$ for some $\delta_j > 0$. Then Assumption 1 holds. Although Assumption 1 is a weak assumption, verifying it is not straightforward (see, e.g., Hong et al. (2011)). In next section, we consider more specific settings and analyze the differentiability. Based on (4), we have the following proposition.

Proposition 1. Suppose that the distribution P_* is a GMM with density (3). Suppose that Assumption 1 is satisfied. Then $\Pr_{\sim P_*} \{c(x, \xi) \leq 0\}$ is differentiable with respect to x , and

$$\nabla_x \Pr_{\sim P_*} \{c(x, \xi) \leq 0\} = \sum_{j=1}^K \pi_j \nabla_x \Pr_{\sim P_j} \{c(x, \xi) \leq 0\}.$$

Proposition 1 shows that the gradient of the probability function with a GMM is the weighted sum of the gradients of the probability functions with the component distributions. Suppose the gradient of the probability function with each component distribution can be handled efficiently. Proposition 1 guarantees that we can handle the gradient for the GMM efficiently. In next section we will also give concrete examples that support the statement.

3. Solution techniques for different scenarios

In this section, we consider several scenarios in which the structured CCPs are used to model various practical decisions. We mainly analyze how the chance constraint can be handled to embrace various optimization techniques.

3.1. JCCPs

We first consider a class of CCPs with the following chance constraint:

$$\Pr_{\sim P_*} \{B\xi \leq x\} \geq 1 - \alpha, \quad (5)$$

where B is a $m \times k$ matrix and x is the m -dimensional decision vector. The constraint (5) is a joint chance constraint given that $m > 1$. Let $b_i^\top$ be the i th row of B and x_i be the i th component of x . By defining $c_i(x, \xi) = b_i^\top \xi - x_i$, we see that (5) is an instance of (1b). In (5) the random vector ξ and the decision vector x are separated. This class of CCPs has been studied widely in the literature, see, e.g., van Ackooij et al. (2010, 2014) and Henrion and Möller (2012). Henrion and Möller (2012) studied (5) and implemented the model in an electricity network capacity optimization problem. They considered the case where P_* is a Gaussian distribution and studied how to estimate the gradient of the probability function in (5). As their main contribution, they showed that under the Gaussian distribution setting, calculations of the gradients can be reduced to calculations of the probability function values.

We now build a similar gradient formula for the probability function with the GMM. To ease the analysis, we define $F^j(x) = \Pr_{\sim P_j} \{B\xi \leq x\}$ and $F^*(x) = \Pr_{\sim P_*} \{B\xi \leq x\}$. Theorem 3.3 (or Theorem 4.1) of Henrion and Möller (2012) provided conditions ensuring the differentiability of $F^j(x)$ and built an expression for $\nabla_x F^j(x)$. In Online Supplement 2, we present their results. We thus take the gradient expression $\nabla_x F^j(x)$ as given. Following Henrion and Möller (2012), we introduce the notion of nondegeneracy for a linear inequality system. Consider the system $Bz \leq x$ with B and x given in (5). Let

$$\mathcal{I}(B, x) = \{I \subset \{1, \dots, m\} \mid \exists z : b_i^\top z = x_i (i \in I), \\ b_i^\top z < x_i (i \in \{1, \dots, m\} \setminus I)\}.$$

The system $Bz \leq x$ is called nondegenerate if $\text{rank}\{b_i\}_{i \in I} = \#I, \forall I \in \mathcal{I}(B, x)$.

Based on Proposition 1, we can derive an expression for the gradient of $F^*(x)$. We summarize the result in the following proposition.

Proposition 2. Suppose P_* is a GMM defined by (3). Let x be such that $Bz \leq x$ is nondegenerate. Then

$$F^*(x) = \sum_{j=1}^K \pi_j F^j(x); \quad \nabla_x F^*(x) = \sum_{j=1}^K \pi_j \nabla_x F^j(x),$$

where $\nabla_x F^j(x)$ is calculated according to Lemma 1 in the Online Supplement.

Proposition 2 generalizes the gradient formulas of Henrion and Möller (2012). Considering that GMMs can model many distribution structures very well (see, e.g., McLachlan and Peel (2000) and Wang and Taaffe (2015)), such a generalization is very useful and can extend the application of the approach proposed by Henrion and Möller (2012). We can use a nonlinear optimization procedure as in Henrion and Möller (2012) to solve the CCP with (5). During the procedure, we can implement Proposition 2 to estimate the function value and the gradient for the chance constraint.

Proposition 2 provides support for accessing the gradient of the probability function that possesses a linear separable structure. In the literature, there have also been investigations concerning the computation of the gradient of the probability function for nonlinear cases with a Gaussian distribution, see, for instance, van Ackooij and Henrion (2014). For such nonlinear cases, we can also implement Proposition 1 to assess the corresponding gradients and to conduct optimization studies.

The idea above can be adopted for another class of CCPs that has the separable structure. The corresponding chance constraint takes the form $\Pr_{\sim P_*} \{\xi \leq f(x)\} \geq 1 - \alpha$, where $f(x) = (f_1(x), \dots, f_k(x))^\top$ with $f_i(x), i = 1, \dots, k$ being real-valued functions defined on X . Let $\xi_{(i)}$ denote the i th component of ξ . By setting $m = k$ and defining $c_i(x, \xi) = \xi_{(i)} - f_i(x)$, the constraint above can be fitted into (1b). The chance constraint was discussed in Prékopa (2003), Henrion (2012) and Henrion and Möller (2012). Under the Gaussian distribution $\mathcal{N}(z|\mu, \Sigma)$ with Σ being positive definite, it has been shown that calculations of $\nabla_z \Pr_{\mathcal{N}(z|\mu, \Sigma)} \{\xi \leq z\}$ can be reduced to calculation of the probability function values. Suppose that $f(x)$ is continuously differentiable in x and P_* is a GMM defined by (3). Then $\Pr_{\sim P_*} \{\xi \leq f(x)\}$ is differentiable in x . Similarly, as in Proposition 2, we can convert the evaluations of the function values and the gradients for the probability function under the GMM to those under the Gaussian distribution. Then, the gradient-based nonlinear optimization approach can be implemented.

3.2. SCCP

We next consider a class of SCCPs with a linear product structure:

$$\text{minimize}_{x \in X} \quad h(x) \quad (6a)$$

$$\text{subject to } \Pr_{\sim P_*} \{f_0(x) + f(x)^\top \xi \leq 0\} \geq 1 - \alpha, \quad (6b)$$

where $f(x) = (f_1(x), \dots, f_k(x))^\top$ and $f_i(x), i = 0, 1, \dots, k$ are real-valued functions defined on X . Constraint (6b) is also an instance of (1b), obtained by setting $m = 1$ and $c_1(x, \xi) = f_0(x) + f(x)^\top \xi$. In contrast with the separable structure considered in the preceding subsection, the random vector ξ and the decision vector x in (6b) take a product form. This class of chance constraints has been widely studied in the literature, see, e.g., Nemirovski and Shapiro (2006), Chen et al. (2010), Zymmler et al. (2013) and Hanasusanto et al. (2017). In our numerical experiments, we will study a typical mean Value-at-Risk (VaR) portfolio selection model that can be written as an instance of (6a)-(6b). It is typically difficult to derive some deterministic representation for the chance constraint (6b). However, when the underlying distribution P_* falls in the normal distribution family, (6b) admits a simplified representation.

Specifically, suppose that the distribution P_* is $\mathcal{N}(z|\mu, \Sigma)$. Then (6b) is equivalent to the following constraint:

$$\Phi^{-1}(1 - \alpha) \sqrt{f(x)^\top \Sigma f(x)} + (f_0(x) + f(x)^\top \mu) \leq 0, \quad (7)$$

where $\Phi^{-1}(\cdot)$ is the inverse of the standard normal distribution function, see, e.g., Henrion (2007). Since it is easy to evaluate $\Phi^{-1}(\cdot)$, (7) can be treated as a deterministic constraint. Suppose further that $h(x)$, $f_0(x)$ and $f(x)$ are linear in x and $\alpha \leq 0.5$. Then constraint (7) is known as a second-order cone constraint. In this setting, Problem (6a)-(6b) can be reformulated as a Second-Order Conic Program (SOCP), which can be solved by conventional optimization softwares, e.g., SeDuMi (Sturm, 1999).

In this subsection, we explore the structures of constraint (6b) under the GMM. Suppose that the distribution P_* takes the form of (3). Following from (4), (6b) is the same as

$$\sum_{j=1}^K \pi_j \Pr_{\sim P_j} \{f_0(x) + f(x)^\top \xi \leq 0\} \geq 1 - \alpha, \quad (8)$$

where P_j is $\mathcal{N}(z|\mu_j, \Sigma_j)$. Here we exclude the case where $f(x) = 0$, because when $f(x) = 0$, the constraint in the probability function does not involve any randomness. Then, the constraint above can be further written as

$$\sum_{j=1}^K \pi_j \Phi \left(\frac{-[f_0(x) + f(x)^\top \mu_j]}{\sqrt{f(x)^\top \Sigma_j f(x)}} \right) \geq 1 - \alpha \quad (9)$$

where $\Phi(\cdot)$ is the standard normal distribution function.

3.2.1. A gradient-based algorithm

Because we reformulate (6b) as a deterministic constraint (9), it is natural to implement a gradient-based optimization

algorithm. We assume that $f_0(x)$ and $f(x)$ are continuously differentiable in x . Let $u_j(x) = f_0(x) + f(x)^\top \mu_j$ and $\tilde{u}_j(x) = \sqrt{f(x)^\top \Sigma_j f(x)}$. Note that the gradient (derivative) of $\Phi(\cdot)$ is simply the density function, denoted as $\phi(\cdot)$, of the standard normal distribution. Then the left-hand side of (9) is differentiable in x and the gradient is

$$\sum_{j=1}^K \pi_j \phi \left(\frac{-u_j(x)}{\tilde{u}_j(x)} \right) \frac{-\nabla u_j(x) \tilde{u}_j(x) + u_j(x) \nabla \tilde{u}_j(x)}{\tilde{u}_j^2(x)}.$$

Therefore, the values and gradients of the function in (9) can be evaluated efficiently. It then allows us to use conventional gradient-based nonlinear optimization algorithms.

A recent study by Peña-Ordieres *et al.* (2020) investigated a quantile reformulation of the CCP. Peña-Ordieres *et al.* (2020) transformed the chance constraint into a quantile (or a VaR) constraint. They then considered a sample approximation and a smooth approximation to construct a smooth constraint to approximate the quantile constraint, and then implemented a nonlinear optimization procedure to solve the approximation problem. Their findings are that in terms of numerical computation, the quantile reformulation shows certain superiority. Recall that a $1 - \alpha$ VaR of a random loss η with distribution P_* is defined as $\text{VaR}_{1-\alpha, P_*}(\eta) := \inf \{v \in \mathcal{R} : \Pr_{\sim P_*} \{\eta \leq v\} \geq 1 - \alpha\}$. Following the idea of Peña-Ordieres *et al.* (2020), we can solve the following VaR-constrained equivalent formulation for Problem (6a)-(6b):

$$\text{minimize}_{x \in X} h(x) \quad (10a)$$

$$\text{subject to } \text{VaR}_{1-\alpha, P_*}(f_0(x) + f(x)^\top \xi) \leq 0. \quad (10b)$$

Let $q(x) = \text{VaR}_{1-\alpha, P_*}(f_0(x) + f(x)^\top \xi)$. We show that under our GMM and linear product setting, we can directly derive the gradient of $q(x)$. Following from the definition of quantile (or VaR), we have:

$$\sum_{j=1}^K \pi_j \Phi \left(\frac{q(x) - [f_0(x) + f(x)^\top \mu_j]}{\sqrt{f(x)^\top \Sigma_j f(x)}} \right) = 1 - \alpha. \quad (11)$$

Given x , since $\Phi(\cdot)$ is strictly increasing, we can evaluate $q(x)$ using a bisection search method. To derive the gradient of $q(x)$, we differentiate both sides of (11) with respect to x and apply the implicit function theorem. This yields

$$\nabla q(x) = \left[\sum_{j=1}^K \pi_j \phi \left(\frac{q(x) - u_j(x)}{\tilde{u}_j(x)} \right) \right]^{-1} g(x)$$

where

$$g(x) = \sum_{j=1}^K \pi_j \phi \left(\frac{q(x) - u_j(x)}{\tilde{u}_j(x)} \right) \times \left(\frac{\nabla u_j(x) \tilde{u}_j(x) + (q(x) - u_j(x)) \nabla \tilde{u}_j(x)}{\tilde{u}_j^2(x)} \right).$$

Since we can evaluate $q(x)$ and $\nabla q(x)$, we can now implement a nonlinear optimization procedure to solve Problem

(10a)-(10b). Note that Peña-Ordieres *et al.* (2020) considered a very general setting, and thus, they need to use sample approximation and smooth approximation. In our setting, we first conduct an input modeling using GMM. Then for the more specific GMM and linear product setting, our problem becomes much easier to handle.

3.2.2. A spatial B&B algorithm

Due to the non-convexity of CCPs, nonlinear optimization procedures typically could only guarantee stationary points for the CCPs. However, the structure of (9) offers some potential for searching for global optimal solutions for the CCPs. In the rest of this section, we explore the structure of the CCP under the GMM and develop a global optimization procedure. We first build the following result. The proof is included in the Online Supplement.

Proposition 3. Suppose that the distribution P_* is a GMM defined by (3). Then Problem (6a)-(6b) is equivalent to the following problem:

$$\text{minimize}_{x \in X, y} h(x) \quad (12a)$$

$$\text{subject to } \Phi^{-1}(y_j) \sqrt{f(x)^\top \Sigma_j f(x)} + (f_0(x) + f(x)^\top \mu_j) \leq 0, \\ j = 1, \dots, K, \quad (12b)$$

$$\sum_{j=1}^K \pi_j y_j \geq 1 - \alpha, 0 \leq y_j \leq 1, j = 1, \dots, K. \quad (12c)$$

The equivalence here guarantees that if (x^*, y^*) is an optimal solution of Problem (12a)-(12c), then x^* is optimal to Problem (6a)-(6b). Problem (12a)-(12c) becomes a deterministic optimization problem with a second-order cone-type structure. A major difficulty for handling the problem is that both x and y are decision variables. To utilize the second-order cone-type structure, one potential approach is to branch the decision variables y .

Note that for the setting, one tractable case is when $K=1$, i.e., the underlying distribution degenerates to a Gaussian distribution. With this setting, as discussed in the preceding subsection, we can transform (12a)-(12c) into a deterministic optimization problem which has constraint (7). In particular, when $h(x)$, $f_0(x)$ and $f(x)$ are all linear in x and $\alpha \leq 0.5$, the resulting problem is an SOCP.

In this subsection, we assume that $f_0(x)$ and $f(x)$ are linear in x , but allow the objective function $h(x)$ to be linear or nonlinear. According to the special structure of the mixture model, we propose a B&B-type procedure to handle Problem (12a)-(12c). We first analyze the bound of the decision vector y for Problem (12a)-(12c) being feasible. Note that for $j = 1, \dots, K$, $1 - \alpha \leq \pi_j y_j + \sum_{i \neq j}^K \pi_i y_i \leq \pi_j y_j + \sum_{i \neq j}^K \pi_i = \pi_j y_j + (1 - \pi_j)$. It follows that $y_j \geq 1 - \alpha/\pi_j$. Let $\underline{y}_j^0 = \max\{0, 1 - \alpha/\pi_j\}$ and $\bar{y}_j^0 = 1$. Thus we only need to consider the region $\Delta^0 := \{y \in \mathcal{R}^K \mid \underline{y}_j^0 \leq y_j \leq \bar{y}_j^0, j = 1, \dots, K\}$.

The basic idea of our B&B algorithm is to partition the region Δ^0 into sub-rectangles and consider a relaxation of

the original problem over a given sub-rectangle. Specifically, consider a sub-rectangle of $\Delta^0 : \Delta = \{y \in \mathbb{R}^K \mid y_j \leq \bar{y}_j, j = 1, \dots, K\}$. Restricting Problem (12a)-(12c) to Δ , we obtain the following problem:

$$\underset{x \in X, y \in \Delta}{\text{minimize}} \quad h(x) \quad (13a)$$

$$\begin{aligned} \text{subject to } & \Phi^{-1}(y_j) \sqrt{f(x)^\top \Sigma_j f(x) + (f_0(x) + f(x)^\top \mu_j)} \leq 0, \\ & j = 1, \dots, K, \end{aligned} \quad (13b)$$

$$\sum_{j=1}^K \pi_j y_j \geq 1 - \alpha. \quad (13c)$$

Notice that $\Phi^{-1}(\cdot)$ is an increasing function and $\sqrt{f(x)^\top \Sigma_j f(x)}$ is non-negative. We consider the following relaxation of Problem (13a)-(13c):

$$\underset{x \in X, y \in \Delta}{\text{minimize}} \quad h(x) \quad (14a)$$

$$\begin{aligned} \text{subject to } & \Phi^{-1}(\underline{y}_j) \sqrt{f(x)^\top \Sigma_j f(x) + (f_0(x) + f(x)^\top \mu_j)} \\ & \leq 0, j = 1, \dots, K, \end{aligned} \quad (14b)$$

$$\sum_{j=1}^K \pi_j y_j \geq 1 - \alpha. \quad (14c)$$

At each iteration of our B&B method, the domain of the vector y is partitioned into sub-rectangles. Solving Problem (14a)-(14c) provides a lower bound for the optimal value of Problem (13a)-(13c). If the optimal solution is also feasible to Problem (12a)-(12c), then its objective value provides an upper bound for the optimal value of Problem (12a)-(12c). At each iteration, a new problem (14a)-(14c) on a certain sub-rectangle Δ is solved and a better solution could possibly be identified. For any problem (14a)-(14c) that is infeasible or its optimal value is greater than or equal to the best attained objective value of Problem (12a)-(12c), the corresponding sub-rectangle is removed from further consideration. We then select one of the remaining active sub-rectangles, under some selection criterion, and partition it into two sub-rectangles for further consideration. This process repeats until all the sub-rectangles are removed. The method is now formally described as follows.

Algorithm 1

Step 0 (Initialization): Set $i := 0$, $\Delta^i := \{y \in \mathbb{R}^K \mid y_j^0 \leq \bar{y}_j, j = 1, \dots, K\}$, $v^* := \infty$, $\underline{v}^i := -\infty$, $flag := 0$. Solve Problem (14a)-(14c) with $\Delta := \Delta^i$.

IF it is infeasible, Problem (12a)-(12c) is infeasible, **STOP**;

ELSE, denote its optimal solution and optimal objective value as (x^c, y^c) and v^c , respectively.

IF x^c is not a feasible solution to Problem (12a)-(12c), $(x^i, y^i) := (x^c, y^c)$, $v^i = v^c$, $\underline{v}^i := v^i$, $\Omega := \{\Delta^i, \underline{v}^i\}$. Set $i := i + 1$, **GOTO** Step 1;

ELSE, $v^* := v^c$, $x^* := x^c$, $flag := 1$, x^* is an optimal solution to Problem (12a)-(12c), **STOP**.

END IF

END IF

Step 1:

IF $\Omega \neq \emptyset$, **GOTO** Step 2;

ELSE IF $flag = 0$, Problem (12a)-(12c) is infeasible, **STOP**;

ELSE, x^* is an optimal solution to Problem (12a)-(12c), **STOP**.

END IF

Step 2: Choose and remove instance $\{\Delta^i, \underline{v}^i\}$ from Ω with the smallest $\underline{v}^i$, denoted as $\underline{v}_s^i$. If more than one rectangles has the smallest $\underline{v}^i$, randomly choose one.

Subdivide Δ^i into two rectangles Δ_1^i and Δ_2^i by dividing an edge of Δ^i at its midpoint according to some partition rule. Denote by Δ_1^i the “upper” rectangle that includes the upper corner $(\bar{y}_1, \dots, \bar{y}_K)$ of Δ^i and by Δ_2^i the “lower” rectangle that includes the lower corner $(y_1, \dots, y_K)$ of Δ^i .

IF $\sum_{j=1}^K \pi_j y_j^1 > 1 - \alpha$ where $(y_1^1, \dots, y_K^1)$ is the lower corner of Δ_1^i , set $\underline{v}_2^i := \underline{v}_s^i$, $\Omega := \Omega \cup \{\Delta_2^i, \underline{v}_2^i\}$. Set $i := i + 1$, **GOTO** Step 1;

ELSE IF $\sum_{j=1}^K \pi_j \bar{y}_j^2 < 1 - \alpha$ where $(\bar{y}_1^2, \dots, \bar{y}_K^2)$ is the upper corner of Δ_2^i , solve Problem (14a)-(14c) with $\Delta := \Delta_1^i$.

IF it is infeasible, $\Omega := \Omega$. Set $i := i + 1$, **GOTO** Step 1;

ELSE, denote its optimal solution and optimal objective value as (x^c, y^c) and v^c , respectively.

IF $v^c \geq v^*$, $\Omega := \Omega$. Set $i := i + 1$, **GOTO** Step 1;

ELSE IF $v^c < v^*$ and x^c is not a feasible solution to Problem (12a)-(12c), $(x^i, y^i) := (x^c, y^c)$, $v^i = v^c$, $\underline{v}_1^i := v^i$, $\Omega := \Omega \cup \{\Delta_1^i, \underline{v}_1^i\}$. Set $i := i + 1$, **GOTO** Step 1;

ELSE, $v^* := v^c$, $x^* := x^c$, $flag := 1$, delete from Ω all instances $\{\Delta^i, \underline{v}^i\}$ with $\underline{v}^i \geq v^*$. Set $i := i + 1$, **GOTO** Step 1.

END IF

END IF

ELSE, set $\underline{v}_2^i := \underline{v}_s^i$. Solve Problem (14a)-(14c) with $\Delta := \Delta_1^i$.

IF it is infeasible, $\Omega := \Omega \cup \{\Delta_2^i, \underline{v}_2^i\}$. Set $i := i + 1$, **GOTO** Step 1;

ELSE, denote its optimal solution and optimal objective value as (x^c, y^c) and v^c , respectively.

IF $v^c \geq v^*$, $\Omega := \Omega \cup \{\Delta_2^i, \underline{v}_2^i\}$. Set $i := i + 1$, **GOTO** Step 1;

ELSE IF $v^c < v^*$ and x^c is not a feasible solution to Problem (12a)-(12c), $(x^i, y^i) := (x^c, y^c)$, $v^i = v^c$, $\underline{v}_1^i := v^i$, $\Omega := \Omega \cup \{\Delta_1^i, \underline{v}_1^i\} \cup \{\Delta_2^i, \underline{v}_2^i\}$. Set $i := i + 1$, **GOTO** Step 1;

ELSE, $v^* := v^c$, $x^* := x^c$, $flag := 1$, delete from Ω all instances $\{\Delta^i, \underline{v}^i\}$ with $\underline{v}^i \geq v^*$. Set $i := i + 1$, **GOTO** Step 1.

END IF

END IF

END IF

We provide more details for Algorithm 1. In Step 2, we only solve Problem (14a)-(14c) over upper rectangle Δ_1^i .

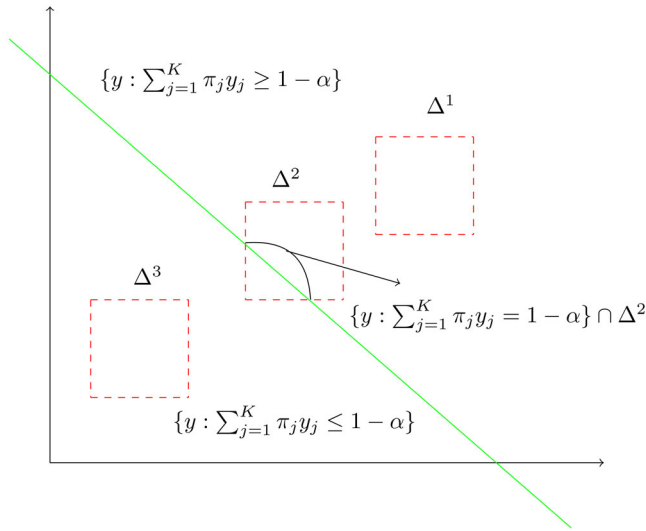


Figure 1. Illustration of Step 2 in Algorithm 1.

Problem (14a)-(14c) over lower rectangle Δ_2^i and the corresponding problem over Δ^i (note that Δ^i is divided to Δ_1^i and Δ_2^i) share the same optimal value (if it exists). The reason is that the lower rectangle Δ_2^i shares the same lower corner with Δ^i so that the constraints (14b) are the same in both problems. Thus we can set $\underline{v}_s^i$ as the lower bound of Δ_2^i , without solving for it.

In Step 2, for the partition rule, we first choose the longest edge among edges whose lower bounds are less than 0.5 and midpoints are larger than 0.5. If we cannot find such an edge, we divide the longest edge among all edges of Δ^i . Furthermore, if more than one edge can be chosen, we break the tie by randomly choosing one of them. Note that in Problem (12a)-(12c), we can obtain an optimal solution by restricting y to the hyperplane $\sum_{j=1}^K \pi_j y_j = 1 - \alpha$. Thus, as in Step 2, we only consider rectangles that intersect with this hyperplane. This can be seen clearly in Figure 1. For the three rectangles in the figure, we only consider the rectangle Δ^2 that intersects with the line (hyperplane) $\sum_{j=1}^K \pi_j y_j = 1 - \alpha$.

Algorithm 1 above can be regarded as a specific implementation of a general B&B method within the framework described by Horst (1986) and Horst *et al.* (1995). Notice that at Step 2, we have $\text{int}\{\Delta_1^i\} \cap \text{int}\{\Delta_2^i\} = \emptyset$ and $\Delta^i = \Delta_1^i \cup \Delta_2^i$, where $\text{int}(S)$ denotes the interior of set S . Algorithm 1 will generate lower bounds for the optimal value of Problem (12a)-(12c). When it generates a feasible solution of Problem (12a)-(12c), the function value of the solution is an upper bound for the optimal value of Problem (12a)-(12c). From the algorithm we can see that the lower bound is tracked by the sequence $\{\underline{v}_s^i\}$ and the upper bound is tracked by v^* . It is easy to see that if Algorithm 1 terminates after a finite number of iterations at Step 1, then it achieves an optimal solution to Problem (12a)-(12c) or verifies the infeasibility of Problem (12a)-(12c). The following proposition gives the global convergent property of Algorithm 1. The proof is included in the Online Supplement.

Proposition 4. Suppose that X is compact, and the functions $h(x)$, $f_0(x)$ and the vector-valued function $f(x)$ are continuous and bounded on X . Suppose that x_s^i produces $\underline{v}_s^i$, i.e., $\underline{v}_s^i = h(x_s^i)$. Denote the optimal value of Problem (12a)-(12c) by v_{opt} . If Algorithm 1 generates an infinite sequence $\{\underline{v}_s^i\}$, then $\underline{v}_s^i \rightarrow v_{\text{opt}}$, and any accumulation point x^* of $\{x_s^i\}$ is an optimal solution to Problem (12a)-(12c).

Now let us further analyze Algorithm 1. In each iteration, we need to solve a subproblem. The subproblems are still nonlinear optimization problems. Therefore, we need to call on softwares or design appropriate procedures to solve the problems. Note that the functions f_0 and f are all linear in x . Then Problem (14a)-(14c) has a conic program structure. We consider a number of cases, depending on whether the problem is convex. For the convexity we have the following proposition, whose proof is included in the Online Supplement.

Proposition 5. Suppose that $f_0(x)$ and $f(x)$ are linear in x . If $y_j \geq 1/2$ for $j = 1, \dots, K$, then (14b) is a second-order cone constraint. Moreover, if $y_j^0 \geq 1/2, j = 1, \dots, K$, or equivalently, $\pi_j \geq 2\alpha, j = 1, \dots, K$, then (14b) is always a second-order cone constraint.

Suppose that the conditions in Proposition 5 are satisfied. If $h(x)$ is linear in x , Problem (14a)-(14c) is an SOCP. Note that the linear case has been studied widely in the literature and management practice. If $h(x)$ is convex quadratic in x , Problem (14a)-(14c) is a convex problem with quadratic objective and second-order cone constraint. For the two settings we can use conventional softwares, e.g., SeDuMi (Sturm, 1999), to solve Problem (14a)-(14c) efficiently. If $h(x)$ is a more general convex function, Problem (14a)-(14c) is a more general convex problem and a typical software can easily solve it to global optimality. In these cases, Algorithm 1 only needs to solve a sequence of convex optimization problems. Our numerical experiments indicate that Algorithm 1 is typically fast for the convex case. Note that in real applications, α usually takes a small value, e.g., 0.05. Then, it is a common case where $\pi_j \geq 2\alpha, j = 1, \dots, K$, when K is small. In the more general case where convexity cannot be preserved, we can transform Problem (14a)-(14c) to an optimization problem with quadratic constraints. Then, we can use some commercial software, e.g., BARON (see, e.g., Tawarmalani and Sahinidis (2005) and Sahinidis (2017)), to solve the problem globally. It is worthwhile noting that for Problem (12a)-(12c), the number of branch variables, which is the number of mixture components K , is usually very small. Thus the B&B method can often work efficiently.

4. Numerical Illustrations

In this section, we first test our algorithm on a CCP with a linear product form, where the parameters are randomly generated. We then implement our approach to solve a portfolio selection problem with risk control on VaR. For all the experiments, the algorithms are coded in Matlab R2019a

Table 1. Optimal values obtained by the nonlinear optimization algorithm (fmincon) and the B&B algorithm for Problem (15a)-(15b) with $\alpha = 0.05$.

K	d	BB Algorithm			fmincon		
		Ave	Min	Max	Ave	Min	Max
2	50	-1292.313	-1439.261	-1122.957	-266.525	-280.969	-253.745
	80	-1511.231	-1712.534	-1376.577	-412.855	-452.296	-388.139
	100	-1490.221	-1550.129	-1434.198	-489.141	-509.706	-469.895
	200	-1312.489	-1352.622	-1294.549	-993.180	-1016.516	-977.100
3	50	-1299.480	-1380.622	-1174.401	-257.799	-273.639	-243.743
	80	-1494.558	-1578.407	-1438.250	-397.983	-428.166	-371.384
	100	-1527.192	-1610.444	-1435.518	-487.827	-515.314	-467.040
	200	-1305.101	-1319.046	-1276.179	-964.811	-983.476	-939.194
4	50	-1218.152	-1360.937	-1075.455	-255.575	-272.517	-246.051
	80	-1519.981	-1643.553	-1425.657	-393.424	-421.265	-374.130
	100	-1499.559	-1528.357	-1463.694	-485.954	-497.402	-481.082
	200	-1301.342	-1329.682	-1292.973	-955.121	-972.577	-945.993
5	50	-1189.390	-1269.474	-1076.447	-250.423	-266.229	-234.681
	80	-1457.467	-1523.690	-1391.711	-384.544	-390.050	-381.163
	100	-1512.267	-1551.571	-1483.883	-484.757	-490.415	-480.607
	200	-1305.836	-1335.323	-1267.842	-951.073	-969.243	-920.881

Table 2. Computational times of the nonlinear optimization algorithm (fmincon) and the B&B algorithm for Problem (15a)-(15b) with $\alpha = 0.05$.

K	d	BB Algorithm			fmincon		
		Ave (s)	Min (s)	Max (s)	Ave (s)	Min (s)	Max (s)
2	50	0.485	0.376	0.846	3.963	3.782	4.235
	80	0.534	0.449	0.730	3.000	2.848	3.118
	100	0.765	0.538	0.971	2.990	2.687	3.243
	200	2.054	1.702	2.549	2.152	2.031	2.294
3	50	2.561	1.855	3.949	4.252	4.012	4.543
	80	4.261	2.636	6.855	3.637	3.379	4.011
	100	5.311	4.030	6.442	3.294	3.128	3.645
	200	18.773	12.816	23.027	2.622	2.588	2.659
4	50	7.655	5.546	11.165	5.339	4.840	5.849
	80	22.951	20.400	27.588	3.966	3.556	4.260
	100	27.338	14.268	36.606	3.733	3.577	3.848
	200	109.866	77.183	152.229	2.986	2.897	3.084
5	50	23.994	12.730	34.068	5.864	5.214	6.175
	80	90.261	29.365	124.705	4.507	4.329	4.774
	100	127.889	92.362	179.351	4.518	4.094	4.862
	200	546.763	361.746	694.797	3.475	3.325	3.636

and run on an Intel Core i5 PC with 3.31 GHz and 8 GB RAM. For the B&B algorithm, we use SeDuMi 1.05 to solve the subproblems if they are SOCPs. If the subproblems are not SOCPs, we call the software BARON 17.10.16. (see, for example, Tawarmalani and Sahinidis (2005) and Sahinidis (2017)) to solve them.

4.1. A test problem

We consider a specific instance of Problem (6a)-(6b). As discussed in Section 3.2, Problem (6a)-(6b) can be transformed as

$$\underset{x \in X}{\text{minimize}} \quad h(x) \quad (15a)$$

$$\text{subject to} \quad \sum_{j=1}^K \pi_j \Phi \left(\frac{-[f_0(x) + f(x)^\top \mu_j]}{\sqrt{f(x)^\top \Sigma_j f(x)}} \right) \geq 1 - \alpha. \quad (15b)$$

In this specific instance, we set $h(x)$ to be a linear function $c^\top x$, X to be $[0, 5]^d$, $f_0(x)$ to be a constant function, and

Table 3. Optimal values obtained by the nonlinear optimization algorithm (fmincon) and the B&B algorithm for Problem (15a)-(15b) with $\alpha = 0.1$.

K	d	BB Algorithm			fmincon		
		Ave	Min	Max	Ave	Min	Max
2	20	-562.368	-651.724	-485.280	-183.016	-207.276	-144.594
	30	-595.069	-657.518	-553.586	-215.998	-255.454	-191.353
	40	-626.106	-665.585	-599.190	-262.620	-273.820	-245.549
	50	-633.679	-680.523	-604.856	-296.914	-317.853	-278.201
3	20	-494.604	-595.203	-424.636	-165.676	-185.190	-143.733
	30	-570.670	-613.571	-490.367	-189.211	-197.973	-179.291
	40	-625.063	-703.532	-567.163	-238.498	-260.696	-210.017
	50	-603.485	-640.659	-571.678	-270.403	-279.597	-255.368
4	20	-520.069	-546.163	-481.660	-153.533	-174.910	-138.101
	30	-570.632	-656.081	-529.852	-189.754	-213.799	-176.238
	40	-613.218	-656.723	-545.359	-227.440	-245.802	-208.123
	50	-613.445	-642.136	-585.575	-271.284	-282.225	-261.711
5	20	-504.371	-587.385	-441.662	-169.846	-175.948	-166.552
	30	-580.474	-654.405	-518.557	-194.949	-202.723	-187.098
	40	-601.461	-675.396	-502.529	-233.970	-248.218	-212.573
	50	-574.048	-647.547	-537.995	-267.855	-303.001	-241.142

$f(x) = (f_1(x), \dots, f_k(x))^\top$ to be a linear function vector $a + Bx$ where $a = (0, \dots, 0)^\top$ and B is an identity matrix. In the experiment, we randomly generate the coefficients of the objective function $h(x)$ and the parameters of the GMM as follows: (i) the elements of c are generated from the $[-10, 0]$ uniform distribution independently; (ii) the elements of μ_j ($j = 1, \dots, K$) are respectively generated from the $[0, 5]$ uniform distribution independently; (iii) Σ_j ($j = 1, \dots, K$) are generated independently from $RR^\top$ where R is a k -dimensional column vector and the elements of R independently follow the $[0, 1]$ uniform distribution; (iv) $\pi_j = r_j / (\sum_{i=1}^K r_i)$ ($j = 1, \dots, K$) where r_j is generated independently from the $[0, 1]$ uniform distribution. We consider two scenarios where $\alpha = 0.05$ and $\alpha = 0.1$. For $\alpha = 0.05$, we set $f_0(x) = -2000$ and ensure that $\pi_j \geq 2\alpha, j = 1, \dots, K$ such that all the subproblems are convex, whereas for $\alpha = 0.1$ we set $f_0(x) = -500$ and make sure that $\pi_j \geq 2\alpha$ is violated for at least one j such that some of the subproblems are nonconvex. For each combination of d and K we randomly generate the input parameters above five times. For the simulated data, we use both the nonlinear optimization algorithm (Matlab fmincon function) and the B&B algorithm (Algorithm 1) to solve Problem (15a)-(15b), respectively. The aim is to learn the features and properties of both algorithms.

For the nonlinear optimization approach, we choose to solve the quantile reformulation. As in Section 3.2, we embed the gradient $\nabla q(x)$ in fmincon to solve the problem. For fmincon, we set the maximal number of iterations to be 10^4 and the tolerance on the constraint violation to be 10^{-4} . The B&B algorithm is terminated if $|U - L|/|L| \leq 10^{-4}$ where U and L are respectively the incumbent upper bound and lower bound for Problem (15a)-(15b). In order to speed up the B&B algorithm, we prune the subproblems for which $\|\bar{y} - y\| < 10^{-2}$ where $\|\cdot\|$ denotes the Euclidean norm. For BARON, we set the relative termination tolerance to be 10^{-3} , which can ensure the accuracy of our solutions when objective function values range from $[100, 2000]$. We also set the parameters “DeltaTerm” and “DeltaA” in BARON to a

Table 4. Computational time of the nonlinear optimization algorithm (fmincon) and the B&B algorithm for Problem (15a)-(15b) with $\alpha = 0.1$.

K	d	BB Algorithm			fmincon		
		Ave (s)	Min (s)	Max (s)	Ave (s)	Min (s)	Max (s)
2	20	8.254	5.874	11.442	8.746	7.808	10.481
	30	33.101	2.490	59.841	7.678	7.139	8.089
	40	137.464	4.791	238.588	6.773	6.299	7.719
	50	236.022	157.798	351.724	6.221	5.639	7.546
3	20	40.717	0.365	141.991	9.913	9.233	11.015
	30	93.066	37.342	239.965	8.751	7.958	9.237
	40	205.304	10.130	616.964	7.431	7.214	7.579
	50	841.691	103.863	3379.935	7.272	6.732	8.044
4	20	60.077	1.353	181.429	10.855	10.350	11.389
	30	294.756	57.802	693.769	10.077	9.058	10.585
	40	1792.053	129.210	3204.703	8.138	7.412	8.597
	50	6849.018	1032.913	10,101.582	8.499	7.826	10.862
5	20	71.852	5.657	217.256	12.942	12.199	14.130
	30	6255.652	2272.214	10,016.737	11.346	10.969	11.632
	40	6061.604	1908.091	10,041.090	9.887	9.449	10.384
	50	20,070.693	20,013.265	20,162.207	9.424	8.617	9.976

value of one, so that BARON will terminate if the value of the optimal value is not improved by at least one for a period of time set by default in BARON. The remaining parameters are set by default in BARON. We initialize BARON and the fmincon function with the same starting point $(0.5, \dots, 0.5)^T$.

Tables 1–4 present some primary numerical results. The optimal values under $\alpha = 0.05$ and $\alpha = 0.1$ derived by the two procedures are reported in Tables 1 and 3, where the three columns “Ave”, “Min” and “Max” represent the average, minimal and maximal optimal values among the five runs obtained by the fmincon function and the B&B algorithm under each combination of K and d . From Tables 1 and 3, we can see that the values returned by the fmincon function are not globally optimal. The B&B algorithm can significantly improve the values to the global optimality. Tables 2 and 4 report the computational times for both algorithms to obtain the corresponding optimal values. The tables show that the computational time of the fmincon function is relatively stable, whereas the computational time of the B&B algorithm increases with K and d . For the case where $\alpha = 0.05$, we have ensured that all the subproblems are convex. Table 2 suggests that we can solve large scale problems with relatively short times using the B&B algorithm. This shows the efficiency of the B&B algorithm in such a case. However, for $\alpha = 0.1$, some of the subproblems may be nonconvex. The computational time of the B&B algorithm significantly increases especially when $K \geq 4$ and $d \geq 40$. It takes more time for BARON to solve a nonconvex problem to optimality than SeDuMi to solve a convex problem to optimality, especially when K and d become large. Therefore, the two algorithms have different merits and can be complementary to each other. In practical applications, it is often better to run B&B starting from the nonlinear optimization algorithm. That is, we use the nonlinear optimization algorithm to find a good local optimal solution, and then use B&B to improve upon the local solution.

We also compare the performances of the B&B algorithm and SAA in solving Problem (15a)-(15b). Similar to

Pagnoncelli *et al.* (2009), we reformulate the SAA of Problem (6a)-(6b) as the following Mixed-Integer Program (MIP)

$$\text{minimize } h(x) \quad (16a)$$

$$\text{subject to } f_0(x) + f(x)^T \xi_i \leq Mz_i, \quad i = 1, \dots, N, \quad (16b)$$

$$\sum_{i=1}^N z_i \leq N\alpha, \quad (16c)$$

$$x \in X, z_i \in \{0, 1\}, \quad i = 1, \dots, N, \quad (16d)$$

where

$$M = \max_{x \in X, i \in \{1, \dots, N\}} f_0(x) + f(x)^T \xi_i$$

with $\{\xi_i\}_{i=1}^N$ generated from the GMM. For the reformulation, the number of auxiliary 0-1 decision variables and the number of constraints are proportional to the sample size. We set $h(x)$ to be a convex quadratic function $x^T Hx + c^T x$ and X to be $[-5, 5]^d$. We randomly generate the coefficients of the objective function $h(x)$ as follows: (i) the elements of c are generated from the $[-100, 0]$ uniform distribution independently; (ii) the matrix H is generated from AA^T where A is a $d \times d$ matrix of independent random numbers. We set $f_0(x) = -150, -100$ corresponding to $\alpha = 0.05, 0.1$, respectively. The remaining parameter settings of Problem (15a)-(15b) and the B&B algorithm are the same as above. We use CPLEX 12.8 to solve Problem (16a)-(16d) with parameters “mip.tolerances.mipgap” and “mip.tolerances.absmipgap” of CPLEX being 10^{-3} so that the optimization will terminate when the absolute or relative MIP gap tolerance is reached. In addition, as long as the computational time of the B&B algorithm and SAA is larger than 3600 seconds after an iteration, we terminate its implementation.

For the SAA, setting the sample size is a challenging problem. Since we consider random distributions whose support is unbounded, the guidelines in Luedtke and Ahmed (2008) may not be used. In this article, we adopt some appropriate sample sizes and study their properties. Specifically for $\alpha = 0.05$, the sample size N is set to be 100, 3000, 5000 and 8000, corresponding to $d = 50, 80, 100$ and 200, respectively, and for $\alpha = 0.1$, N is set to be 100, 300, 500 and 2000, corresponding to $d = 20, 30, 40$ and 50, respectively. To evaluate the performance of the SAA under the given sample size, we repeatedly implement the SAA five times under each combination of K and d . In Tables 5 and 7 in the Online Supplement, we report the optimal values derived by the two procedures under $\alpha = 0.05$ and $\alpha = 0.1$. The column h^* represents the optimal values obtained by the B&B algorithm and the three columns “Ave”, “Min” and “Max” represent the average, minimal and maximal optimal values among the five runs. If the SAA fails to return a feasible solution of Problem (15a)-(15b), we discard that solution when reporting the results in Tables 5 and 7 in the Online Supplement. In Tables 6 and 8 in the Online Supplement, we report the computational time for both algorithms to reach the termination criterion. We denote by “N” for the case where the B&B algorithm or the SAA fails

to obtain a feasible solution of Problem (15a)-(15b) within the time limit.

From the experiments, we can observe that the SAA has a stability issue, which coincides with the findings in the literature (see, for example, Pagnoncelli *et al.* (2009) and Peña-Ordieres *et al.* (2020)). In Table 5 in the Online Supplement, there are a number of combinations (with “N”) where the SAA did not find feasible solutions for the five replications. For other rows where there are values, we may also encounter infeasibility for some of the replications. This suggests that the study on the sample size determination for SAA is important. For the case where $\alpha = 0.05$, Tables 5 and 6 in the Online Supplement suggest that when the dimension d is high, e.g., $d = 100, 200$, and the component number K is not very large, e.g., $K = 2, 3, 4$, the B&B algorithm can often obtain smaller objective function values than the SAA within a relatively shorter time. For the case where $\alpha = 0.1$, SAA still admits some instability in our experiments, but the issue is much less severe. Table 7 in the Online Supplement suggests that the performances of the B&B algorithm and SAA are close to each other in terms of the optimality. However, Table 8 in the Online Supplement shows that the computational time of the B&B algorithm is much longer than that of the SAA, especially when both d and K become large. This may be due to that with $\alpha = 0.1$ some of the subproblems (14a)-(14c) become nonconvex, and thus are more difficult to solve. The computational results suggest that the B&B algorithm may be competitive in terms of the stability. It may also enjoy a computational efficiency when the confidence level is small.

Furthermore, we show that the B&B algorithm can also handle Problem (15a)-(15b) with a more general convex objective function. Note that this problem may not be handled by SAA. Specifically, we set $h(x) = x^\top Hx + \sum_{i=1}^d \exp(5 - x_i)$. Here, we let $f_0(x) = -50$, $d = 30$, $\alpha = 0.01$ and $K = 3$, and solve all the subproblems (14a)-(14c) using BARON. The remaining input parameters are the same as the ones above. The performances of the B&B algorithm within four replications are shown in Figure 4 in the Online Supplement where the relative error is defined as $|U - L|/|L|$. When the B&B algorithm has not found feasible solutions until the current iteration, we let the relative error be one. The figure suggests that throughout the four replications, the B&B algorithm can solve the problem to optimality quickly, as the relative error will reduce to zero after a few iterations.

4.2. A portfolio optimization problem

In this subsection, we consider a portfolio optimization problem with the following form:

$$\text{maximize}_{x \in \mathcal{R}_+^d} \quad E[r^\top x] \quad (17a)$$

$$\text{subject to} \quad \text{VaR}_{1-\alpha, P_*}(-r^\top x) \leq \nu, \quad (17b)$$

$$\mathbf{1}^\top x = 1, \quad (17c)$$

where the portfolio consists of d assets whose return rates are d -dimensional random vector $r = (r_1, \dots, r_d)^\top$ following a

distribution P_* , x is the d -dimensional decision vector consisting of the percentile proportions of each asset, and $\text{VaR}_{1-\alpha}(\cdot)$ denotes the VaR of a random variable at confidence level $1 - \alpha$. The total wealth is set to one. In addition, we prohibit the short selling of assets by restricting x to $\mathcal{R}_+^d$. The objective of this problem is to maximize the expected total return rate of the portfolio while controlling the risk (VaR of the portfolio) below a certain threshold ν . It is clear that Problem (17a)-(17c) can be reformulated as the following CCP:

$$\text{maximize}_{x \in \mathcal{R}_+^d} \quad E[r^\top x] \quad (18a)$$

$$\text{subject to} \quad \Pr_{\sim P_*} \{-r^\top x \leq \nu\} \geq 1 - \alpha, \quad (18b)$$

$$\mathbf{1}^\top x = 1. \quad (18c)$$

4.2.1. Constructing the portfolio

In our experiments, we construct a hedge fund portfolio consisting of 12 indices of Dow Jones Credit Suisse Broad Hedge Fund, which are listed in Table 9 in the Online Supplement. Therefore, the dimension d is set to 12. We collect 273 monthly returns of these 12 indices from April 1994 to December 2016, and use the data to model the distribution P_* . We first report some descriptive statistics of the collected data in Table 10 in the Online Supplement. From Table 10 in the Online Supplement we can see that the skewness of the majority of the return rates is negative, which is significantly different from a zero skewness of the normal distribution. Meanwhile, most of the asset return rates have kurtosis greater than three, indicating that the underlying distributions of the monthly return rates are more outlier-prone than the normal distribution.

Therefore, it is intuitively not appropriate to fit the distribution P_* to Gaussian distributions. In the experiments, we use some GMM to fit the data. The detailed implementation of the EM algorithm for the financial data is introduced in Online Supplement 8. For the GMM derived, we further analyze some of the marginal distributions. We consider three hedge funds that have the largest absolute values of skewness and kurtosis, i.e., Convertible arbitrage, Event driven distressed and Fixed income arbitrage. We consider the three marginal distributions for the fitted Gaussian distribution and GMM. Figure 5 in the Online Supplement displays the corresponding quantile plots for the fitted Gaussian distribution, where the horizontal axis is the sample quantile based on a set of samples with sample size 1000 simulated from the Gaussian distribution, and the vertical axis is the empirical quantile of the real data. In contrast, Figure 6 in the Online Supplement displays the quantile plots for the fitted GMM. Note that in the quantile plots, we have eliminated the highest 2.5% and lowest 2.5% of both real data and simulated data so that the extreme returns can be avoided. From the plots we can see that GMM provides a superior fit than the Gaussian distribution. Finally, we remark that using GMM to fit the financial data has been advocated in the literature, see, e.g., Buckley *et al.* (2008).

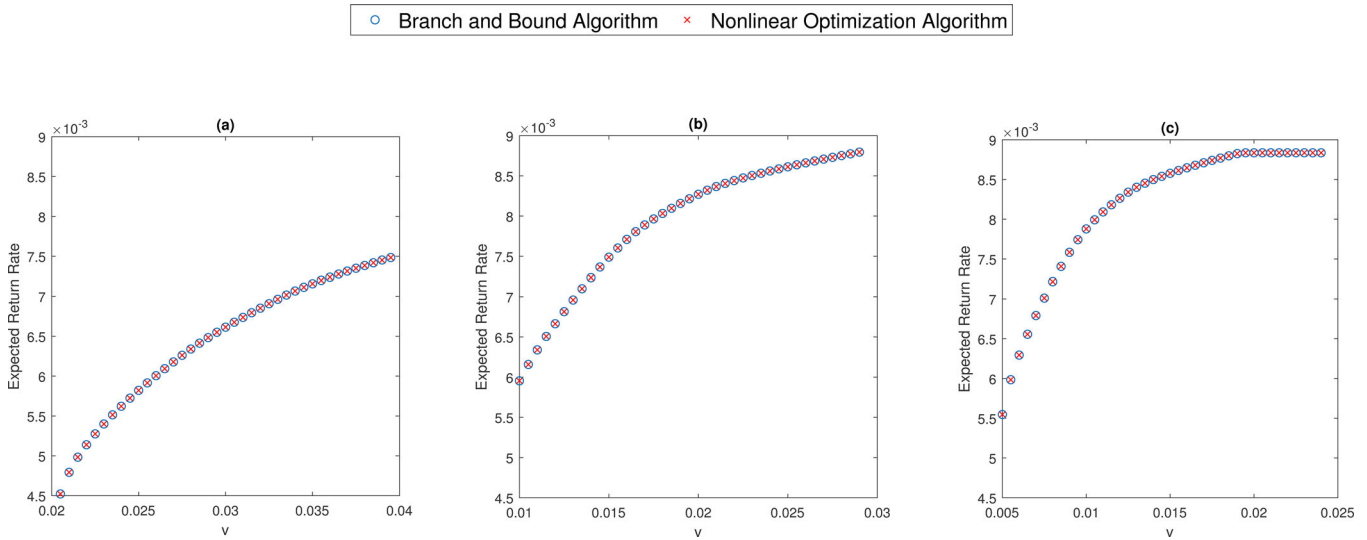


Figure 2. Risk-return pairs for Problem (17a)-(17c) when (a) $\alpha = 0.01$, (b) $\alpha = 0.05$ and (c) $\alpha = 0.1$.

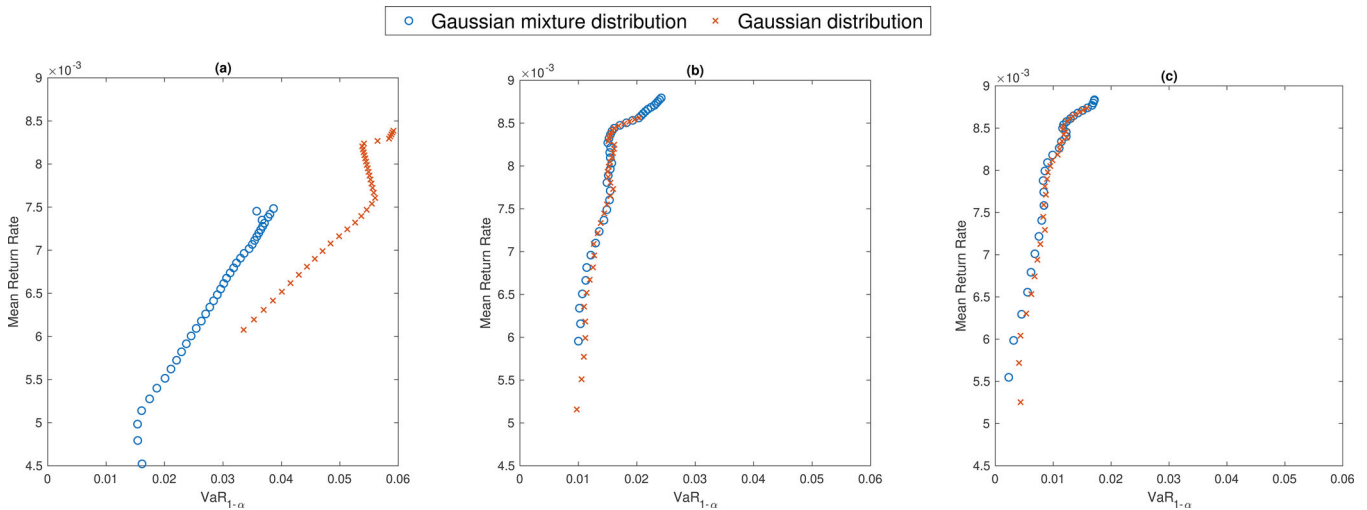


Figure 3. Historical risk-return pairs under different settings: GMM versus Gaussian distribution when (a) $\alpha = 0.01$, (b) $\alpha = 0.05$ and (c) $\alpha = 0.1$.

4.2.2. Computational results

With the distribution specified, we now implement the nonlinear optimization algorithm and the B&B algorithm to handle the problem. Similarly, we let $q(x) = \text{VaR}_{1-\alpha, P_*}(-r^T x) - v$. It can be verified that $\nabla q(x)$ has the following closed form:

$$\nabla q(x) = \left[\sum_{j=1}^K \frac{\pi_j \phi\left(\frac{q(x) + x^T \mu_j}{\sqrt{x^T \Sigma_j x}}\right)}{\sqrt{x^T \Sigma_j x}} \right]^{-1} \sum_{j=1}^K \pi_j \phi\left(\frac{q(x) + x^T \mu_j}{\sqrt{x^T \Sigma_j x}}\right) \times \left(-\frac{\mu_j}{\sqrt{x^T \Sigma_j x}} + \frac{(q(x) + x^T \mu_j) \Sigma_j x}{(x^T \Sigma_j x)^{3/2}} \right).$$

We embed $\nabla q(x)$ in `fmincon` to solve Problem (17a)-(17c). The parameters of the `fmincon` function are the same as the ones in Section 4.1. In the B&B algorithm, we use the solutions returned by the nonlinear optimization algorithm as the initial upper bound in the B&B algorithm and set the relative termination tolerance to be 10^{-2} . In addition, we set

“DeltaA” in BARON to 10^{-4} . The remaining parameter settings are the same as for the test problem above.

Figure 2 shows the optimal values of Problem (17a)-(17c) obtained by both algorithms under different combinations of v and α . For $\alpha = 0.01$, the value of v ranges from 2% to 4% with the step being 0.05%; for $\alpha = 0.05$, the value of v ranges from 1% to 3% with the step being 0.05%; for $\alpha = 0.1$, the value of v ranges from 0.5% to 2.5% with the step being 0.05%. From Figure 2, we can find that for various combinations of v and α , the nonlinear optimization algorithm based on quantile reformulation can indeed find the optimal solutions, and at the same time, the B&B algorithm verifies the global optimality.

Finally, we briefly compare the performance of the optimal portfolio derived from the GMM and the Gaussian distribution, respectively. We vary the values of v (the values are the same as in Figure 2.) and obtain the corresponding optimal solutions of Problem (18a)-(18c) for these two input models. We then calculate the historical mean return rate and historical VaR corresponding to the optimal solutions.

The result is shown in Figure 3, where the vertical axis and the horizontal axis represent the historical mean return rate and historical VaR, respectively. For each value of v , a “o” or a “x” is plotted in the figure. Note that if the results for different v values coincide, then the “o” or the “x” also coincide, which means the numbers of “o” or “x” may appear different in the figure. From Figure 3, we find that when $\alpha = 0.01$, using the GMM we can obtain a better portfolio frontier. Furthermore, we can find that the empirical VaR has a larger deviation from the imposed constraints for the Gaussian distribution setting. This may suggest that the GMM shows superiority over the Gaussian distribution for capturing the tail risks and for the portfolio selection. Finally, we include some analysis on out-of-sample performances for both GMM and Gaussian distribution in Online Supplement 9, which further shows some features of our approach.

5. Conclusions

In this article, we studied using GMM to model the randomness in CCPs. We analyzed the structures and properties of CCPs with GMM. Based on the results we studied several scenarios and discussed how to handle the CCPs in these scenarios for efficient computation. We also conducted some numerical experiments to demonstrate the implementation of our approach. We expect that the approach proposed in this article could generalize the scope and applicability of CCPs.

Acknowledgments

We thank Andrew Schaefer, the associate editor and two anonymous referees for their effort and valuable comments/suggestions.

Funding

This research was partially supported by the Natural Science Foundation of China (Grants 71722009, 72071148, 71890973, 71721001).

Notes on contributors

Zhaolin Hu is a Professor in School of Economics and Management at Tongji University. He obtained his PhD degree in Industrial Engineering and Logistics Management from the Hong Kong University of Science and Technology, and his BSc degree in Mathematics and Applied Mathematics from Zhejiang University. His current research interests include stochastic optimization, stochastic simulation, machine learning and risk management.

Wenjie Sun is a PhD candidate affiliated with the Department of Analytics and Operations in the NUS Business School at National University of Singapore. His current research interests lie in the revenue management and inventory control.

Shushang Zhu is a Professor in School of Business at Sun Yat-Sen University. He obtained his PhD degree in Management Science and Engineering from Academy of Mathematics and Systems Science, Chinese Academy of Sciences, and his MSc degree in Applied Mathematics and BSc degree in Mathematical Statistics from Xiangtan

University. His current research interests include portfolio optimization, systemic risk measurement and quantitative risk management.

ORCID

Zhaolin Hu  <http://orcid.org/0000-0002-8656-766X>

Wenjie Sun  <http://orcid.org/0000-0003-1155-4335>

Shushang Zhu  <http://orcid.org/0000-0003-3981-6659>

References

- Bremer, I., Henrion, R. and Möller, A. (2015) Probabilistic constraints via SQP solver: Application to a renewable energy management problem. *Computational Management Science*, **12**(3), 435–459.
- Bishop, C.M. (2006) *Pattern Recognition and Machine Learning*, Springer, New York, NY.
- Buckley, I., Saunders, D. and Seco, L. (2008) Portfolio optimization when asset returns have the Gaussian mixture distribution. *European Journal of Operational Research*, **185**(3), 1434–1461.
- Calafiore, G. and Campi, M.C. (2005) Uncertain convex programs: Randomized solutions and confidence levels. *Mathematical Programming*, **102**(1), 25–46.
- Calafiore, G. and Campi, M.C. (2006) The scenario approach to robust control design. *IEEE Transactions on Automatic Control*, **51**(5), 742–753.
- Charnes, A., Cooper, W.W. and Symonds, G.H. (1958) Cost horizons and certainty equivalents: An approach to stochastic programming of heating oil. *Management Science*, **4**, 235–263.
- Chen, W., Sim, M., Sun, J. and Teo, C.P. (2010) From CVaR to uncertainty set: Implications in joint chance constrained optimization. *Operations Research*, **58**, 470–485.
- Chen, Z., Peng, S. and Liu, J. (2018) Data-driven robust chance constrained problems: A mixture model approach. *Journal of Optimization Theory and Applications*, **179**(3), 1065–1085.
- Dentcheva, D., Prkopa, A. and Ruszczyński, A. (2000) Concavity and efficient points of discrete distributions in probabilistic programming. *Mathematical Programming*, **89**(1), 55–77.
- Frühwirth-Schnatter, S. (2006) *Finite Mixture and Markov Switching Models*, Springer, New York, NY.
- Geng, X. and Xie, L. (2019) Data-driven decision making with probabilistic guarantees (part 1): A schematic overview of chance-constrained optimization. arXiv preprint *arXiv:1903.10621*.
- Hanasusanto, G.A., Roitch, V., Kuhn, D. and Wiesemann, W. (2017) Ambiguous joint chance constraints under mean and dispersion information. *Operations Research*, **65**(3), 751–767.
- Henrion, R. (2007) Structural properties of linear probabilistic constraints. *Optimization*, **56**(4), 425–440.
- Henrion, R. (2012) Gradient estimates for Gaussian distribution functions: Application to probabilistically constrained optimization problems. *Numerical Algebra, Control and Optimization*, **2**(4), 655–668.
- Henrion, R. (2013) A critical note on empirical (sample average, Monte Carlo) approximation of solutions to chance constrained programs, in *System Modeling and Optimization*, volume 391 of IFIP Advances in Information and Communication, Springer-Verlag, Berlin, pp. 25–37.
- Henrion, R. and Möller, A. (2012) A gradient formula for linear chance constraints under Gaussian distribution. *Mathematics of Operations Research*, **37**(3), 475–488.
- Hong, L.J., Yang, Y. and Zhang, L. (2011) Sequential convex approximations to joint chance constrained programs: A Monte Carlo approach. *Operations Research*, **59**, 617–630.
- Horst, R. (1986) A general class of branch-and-bound methods in global optimization with some new approaches for concave minimization. *Journal of Optimization Theory and Application*, **51**, 271–291.
- Horst, R., Pardalos, P.M. and Thoai, N.V. (1995) *Introduction to Global Optimization*, Kluwer Academic Publishers, Dordrecht, The Netherlands.

- Hu, Z., Hong, L.J. and Zhang, L. (2013) A smooth Monte Carlo approach to joint chance constrained program. *IEE Transactions*, **45**(7), 716–735.
- Lasserre, J.B. and Weisser, T. (2021) Distributionally robust polynomial chance-constraints under mixture ambiguity sets. *Mathematical Programming*, **185**, 409–453.
- Law, A.M. (2013) *Simulation Modeling and Analysis*, McGraw-Hill, New York, NY.
- Lejeune, M.A. (2012) Pattern-based modeling and solution of probabilistically constrained optimization problems. *Operations Research*, **60**(6), 1356–1372.
- Li, J.Q. and Barron, A.R. (2000) Mixture density estimation. *Advances in Neural Information Processing Systems*, **12**, 279–285.
- Luedtke, J. and Ahmed, S. (2008) A sample approximation approach for optimization with probabilistic constraints. *SIAM Journal on Optimization*, **19**(2), 674–699.
- Luedtke, J., Ahmed, S. and Nemhauser, G.L. (2010) An integer programming approach for linear programs with probabilistic constraints. *Mathematical Programming*, **122**, 247–272.
- Maugis-Rabusseau, C. and Michel, B. (2013) Adaptive density estimation for clustering with Gaussian mixtures. *ESAIM: Probability and Statistics*, **17**, 698–724.
- McLachlan, G.J. and Peel, D. (2000) *Finite Mixture Models*, Wiley, New York, NY.
- Miller, L.B. and Wagner, H. (1965) Chance-constrained programming with joint constraints. *Operations Research*, **13**, 930–945.
- Nemirovski, A. and Shapiro, A. (2006) Convex approximations of chance constrained programs. *SIAM Journal on Optimization*, **17**, 969–996.
- Pagnoncelli, B.K., Ahmed, S. and Shapiro, A. (2009) Sample average approximation method for chance constrained programming: Theory and applications. *Journal of Optimization Theory and Applications*, **142**, 399–416.
- Parzen, E. (1962) On estimation of a probability density function and mode. *The Annals of Mathematical Statistics*, **33**(3), 1065–1076.
- Pearson, K. (1894) Contributions to the mathematical theory of evolution. *Philosophical Transactions of the Royal Society of London, A* **185**, 71–110.
- Peña-Ordieres, A., Luedtke, J. and Wächter, A. (2020) Solving chance-constrained problems via a smooth sample-based nonlinear approximation. *SIAM Journal on Optimization*, **30**(3), 2221–2250.
- Prékopa, A. (2003) Probabilistic programming, in *Stochastic Programming, Handbooks in OR&MS*. Vol. 10, Elsevier, Amsterdam, The Netherlands, pp. 267–351.
- Rosenblatt, M. (1956) Remarks on some nonparametric estimates of a density function. *The Annals of Mathematical Statistics*, **27**, 832–837.
- Sahinidis, N.V. (2017) *BARON 17.8.9: Global Optimization of Mixed-Integer Nonlinear Programs*, User's manual. <https://sahinidis.coe.gatech.edu/baron>
- Sturm, J.F. (1999) Using SeDuMi 1.02, a MATLAB toolbox for optimization over symmetric cones. *Optimization Methods and Software*, **11/12**, 625–653.
- Sun, W., Hu, Z. and Hong, L.J. (2018) Gaussian mixture model-based random search for continuous optimization via simulation in *Proceedings of the 2018 Winter Simulation Conference*, IEEE Press, Piscataway, NJ, pp. 2003–2014.
- Tawarmalani, M. and Sahinidis, N.V. (2005) A polyhedral branch-and-cut approach to global optimization. *Mathematical Programming*, **103**(2), 225–249.
- van Ackooij, W. and Henrion, R. (2014) Gradient formulae for nonlinear probabilistic constraints with Gaussian and Gaussian-like distributions. *SIAM Journal on Optimization*, **24**(4), 1864–1889.
- van Ackooij, W., Henrion, R., Möller, A. and Zorgati, R. (2010) On probabilistic constraints induced by rectangular sets and multivariate normal distributions. *Mathematical Methods of Operations Research*, **71**(3), 535–549.
- van Ackooij, W., Henrion, R., Möller, A. and Zorgati, R. (2014) Joint chance constrained programming for hydro reservoir management. *Optimization and Engineering*, **15**(2), 509–531.
- Wang, J. and Taaffe, M.R. (2015) Multivariate mixtures of normal distributions: Properties, random vector generation, fitting, and as models of market daily changes. *INFORMS Journal on Computing*, **27**(2), 193–203.
- Wilson, R. and Calway, A. (2001) Multiresolution Gaussian mixture models for visual motion estimation, in *International Conference on Image Processing*, IEEE, Thessaloniki, Greece, pp. 921–924.
- Zeevi, A.J. and Meir, R. (1997) Density estimation through convex combinations of densities: Approximation and estimation bounds. *Neural Networks*, **10**(1), 99–109.
- Zymler, S., Kuhn, D. and Rustem, B. (2013) Distributionally robust joint chance constraints with second-order moment information. *Mathematical Programming*, **137**(1–2), 167–198.